

App Development Project Report

1. Student Details

Name: Rishab Arora

Roll Number: 25F2008178

Email: 25f2008178@ds.study.iitm.ac.in

About Me: I'm Rishab Arora, a student doing the BS in Data Science & Applications program at IIT Madras and the BTech CSE Program at Maharaja Surajmal Institute of Technology (GGSIPU), New Delhi.

2. Project Details

Project Title: Trekking Management Application

Problem Statement & Approach:

Adventure organizations require efficient systems to manage trekking activities involving trek organizers, staff, and participants. Currently, many trekking groups rely on spreadsheets, calls, or manual coordination, which makes it difficult to manage trek approvals, participant registrations, slot availability, and booking tracking.

The app was built using Flask as the backend framework with a modular structure. It allows the admin to view all ongoing activities and approve the creation of treks and their staff, while users (trekkers) can easily view all the available treks, create bookings for them and await the status updates.

3. AI/LLM Declaration

No use of AI/LLM.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Vue.JS	Frontend JavaScript framework for building reactive user interfaces
Bootstrap 5	Frontend styling and responsive design

Redis	In-memory data store operating as the message broker for Celery tasks
Celery	Asynchronous task queue for handling background jobs and scheduled operations
MailHog	Development SMTP server and interface for testing email delivery and notifications
Flask-Security	User authentication and session management
SQLite	Lightweight local database for storing user data

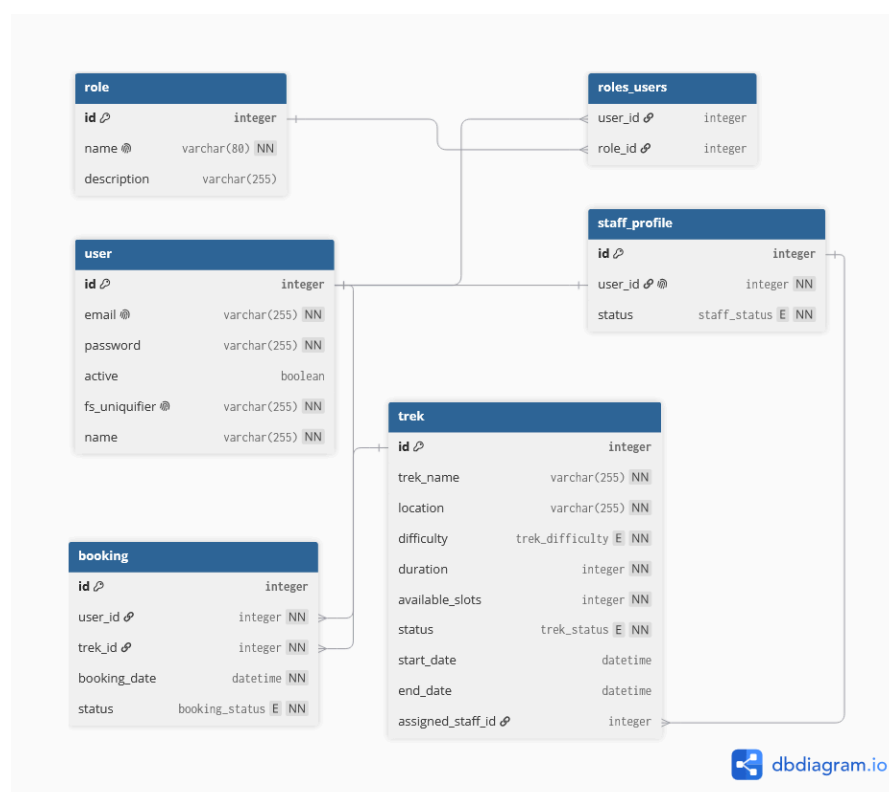
5. Database Schema / ER Diagram

Tables:

1. **User:** stores user credentials and profile details (**id**, **email**, password, active, fs_uniquifier, name).
2. **Role:** defines access permissions for different user types (**id**, **name**, **description**).
3. **StaffProfile:** stores operational status and profiles for trek staff members (id, user_id, status).
4. **Trek:** stores trek details, schedule, availability, and staff assignment (id, trek_name, location, difficulty, duration, available_slots, status, start_date, end_date, assigned_staff_id).
5. **Booking:** logs trekker registrations and booking statuses for specific treks (id, user_id, trek_id, booking_date, status).

Relationships:

- **Many-to-Many** → **User** ↔ **Role** (via **roles_users** join table)
- **One-to-One** → **User** ↔ **StaffProfile**
- **One-to-Many** → **User** → **Booking**
- **One-to-Many** → **StaffProfile** → **Trek**
- **One-to-Many** → **Trek** → **Booking**



6. API Resource Endpoints

Endpoint	Method	Description
User Operations		
<code>/api/user/profile</code>	GET	Retrieve current logged-in trekker profile
<code>/api/user/profile</code>	PUT	Update profile details (name, email, password)
<code>/api/user/treks</code>	GET	Retrieve open trek expeditions available for booking
<code>/api/user/treks/{trek_id}</code>	GET	Retrieve details for a specific open trek
<code>/api/user/bookings</code>	GET	Retrieve all trek bookings made by the current user
<code>/api/user/bookings</code>	POST	Book an available slot on an open trek
<code>/api/user/bookings/{booking_id}</code>	GET	Retrieve booking details by ID
<code>/api/user/bookings/{booking_id}/cancel</code>	PUT	Cancel an active trek booking and release slot back to the trek
<code>/api/user/export</code>	POST	Launch async Celery task to export user booking history to CSV
<code>/api/user/export/{task_id}</code>	GET	Poll status of async export task
<code>/api/user/export/{task_id}/download</code>	GET	Download completed CSV export file
Staff Operations		
<code>/api/staff/treks</code>	GET	Retrieve treks assigned to the current staff guide

<code>/api/staff/treks/{trek_id}</code>	GET	Get details of an assigned trek
<code>/api/staff/treks/{trek_id}</code>	PUT	Update slots or status (Open/Closed/Completed) for assigned trek
<code>/api/staff/treks/{trek_id}/users</code>	GET	Retrieve list of registered participants for an assigned trek
<code>/api/staff/treks/{trek_id}/complete</code>	PUT	Mark trek as completed and update active bookings to completed

Admin Operations

<code>/api/admin/treks</code>	GET	List all trek routes in the system
<code>/api/admin/treks</code>	POST	Create a new trek route in the system
<code>/api/admin/treks/{trek_id}</code>	GET	Retrieve specific trek details for admin
<code>/api/admin/treks/{trek_id}</code>	PUT	Update trek details
<code>/api/admin/treks/{trek_id}</code>	DELETE	Delete trek route
<code>/api/admin/treks/{trek_id}/assign</code>	PUT	Assign or reassign active staff guide to trek
<code>/api/admin/staff</code>	GET	Retrieve staff profiles
<code>/api/admin/staff</code>	POST	Create user with staff role and staff profile
<code>/api/admin/staff/{staff_id}</code>	GET	Retrieve specific staff profile details
<code>/api/admin/staff/{staff_id}</code>	PUT	Update staff profile
<code>/api/admin/staff/{staff_id}/status</code>	PUT	Update staff operational status

<code>/api/admin/users</code>	GET	List registered users filtered by role/status
<code>/api/admin/users/{user_id}/status</code>	PUT	Update user status / roles
<code>/api/admin/search_users</code>	GET	Search registered users by query string
<code>/api/admin/search_staff</code>	GET	Search staff profiles by query string
<code>/api/admin/search_treks</code>	GET	Search trek routes by query string

YAML API Definition File:

Included separately in the submission ZIP as `api.yaml`.

7. Video Presentation

Drive Link:

https://drive.google.com/file/d/1qBZntPRBwqsEOLv2k0rj5ZqjmdXiwfov/view?usp=drive_link